

Experiment 1

Siddhant Bajaj

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. Hash Generation using SHA-256:

- o Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

2. Target Hash Generation with Nonce:

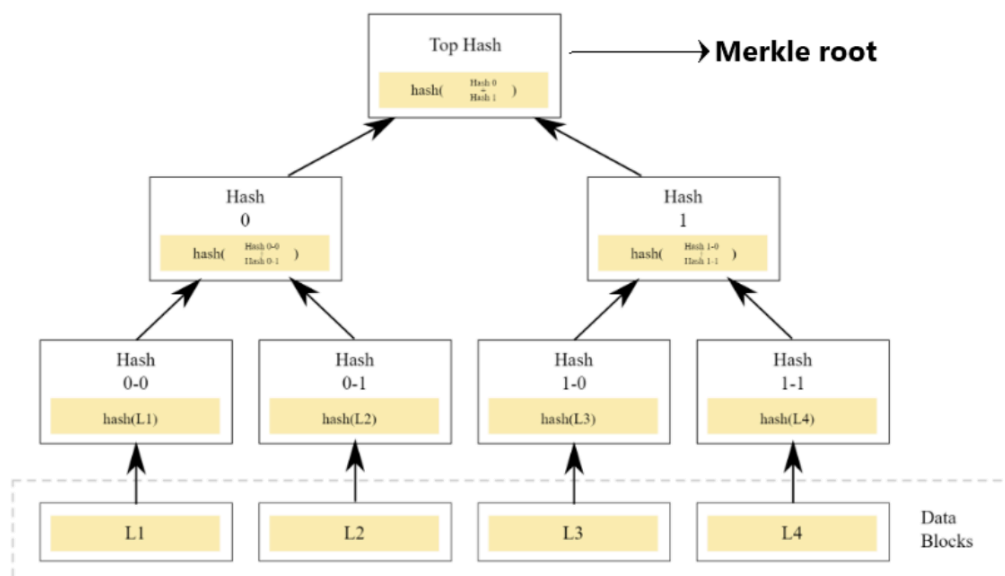
- o Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

3. Proof-of-Work Puzzle Solving:

- o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

4. Merkle Tree Construction:

- o Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.



Program and Output

 bc lab1

Program 1:

```
import hashlib
def create_hash(string):
    hash_object = hashlib.sha256()
    hash_object.update(string.encode('utf-8'))
    hash_string = hash_object.hexdigest()
    return hash_string
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Output:

Enter a string: Siddhant

Hash: 5fda535a5305b3258469c38c70d8c346db74c0c02c1bff94b6fc7dfb31a36597

Program 2:

```
import hashlib
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")
hash_string = input_string + nonce
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()
print("Hash Code:", hash_code)
```

Output:

Enter a string: Siddhant

Enter the nonce: 1022

Hash Code: 58a125640a69cafe3e9c4779657f772ba9770600ff68660c62cd49236a084f3e

Program 3:

```
import hashlib

def find_nonce(input_string, target):
    nonce = 0
    while True:
        hash_string = input_string + str(nonce)
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()
        if hash_code.startswith(target):
            print("Hash:", hash_code)
            return nonce
        nonce += 1

# Get user input
input_string = input("Enter a string: ")
target = input("Enter the required leading zeros: ")

# Find the expected nonce
expected_nonce = find_nonce(input_string, target)

# Display the result
print("Input String:", input_string)
print("Leading Zeros:", target)
print("Expected Nonce:", expected_nonce)
```

Output:

Enter a string: Siddhant

Enter the required leading zeros: 0000

Hash: 00002479b67fb477c0794e565bb1fbadbe5b1eb20fcaa1e9adf637ab814ab295

Input String: Siddhant

Leading Zeros: 0000

Expected Nonce: 9560

Program 4:

```
import hashlib

# Function to generate SHA-256 hash
def hash_data(data):
    return hashlib.sha256(data.encode('utf-8')).hexdigest()

# Function to construct the Merkle Tree
def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    # Hash all transactions before combining them
    transactions = [hash_data(transaction) for transaction in transactions]
    print("\nInitial Transaction Hashes:")

    for i, transaction_hash in enumerate(transactions, 1):
        print(f"T{i}: {transaction_hash}")

    print("\nMerkle Tree Construction:")
    level = 1

    # Continue until only one hash remains
    while len(transactions) > 1:
        # If number of hashes is odd, duplicate the last hash
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []

        # Combine hashes in pairs
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i + 1]

            # Hash the combined hashes
            hash_combined = hash_data(combined)

            # Print intermediate hash
            print(
                f"Level {level}: "
```

```

        f"Hash({transactions[i][:10]}... + "
        f"{transactions[i + 1][:10]}...) = "
        f"{hash_combined}"
    )
    new_transactions.append(hash_combined)
    transactions = new_transactions
    level += 1
    return transactions[0]
# Get number of transactions from user
num_transactions = int(input("Enter the number of transactions: "))
transactions = []
# Get transactions from user
for i in range(num_transactions):
    transaction = input(f"Enter Transaction {i + 1}: ")
    transactions.append(transaction)
# Construct the Merkle Tree
merkle_root = build_merkle_tree(transactions)
# Display the final Merkle Root
print("\nMerkle Root:")
print(merkle_root)

```

Output:

```

Enter the number of transactions: 3
Enter Transaction 1: Siddhant -> Jai : $100
Enter Transaction 2: Siddhant -> Raj : $150
Enter Transaction 3: Raj -> Jai : $300
Initial Transaction Hashes:
T1: 8e32c5506f427f4d6f04857cf025cc33695d6ddf941ca83706ad81d28c03df3d
T2: d18570a3f7f859d64b156ca6e9ccedf3ca2150985eff098ba957c952650ae526
T3: 3a2c2ce298ab1372a2798a22f8491d8736923b93aa2473082c0895a6512396a5

```

Merkle Tree Construction:

Level 1: Hash(8e32c5506f... + d18570a3f7...) =

1a6ad2633753924cbc93037790fd24de325abb1c7265ab92aa3f95df3c180e2b

Level 1: Hash(3a2c2ce298... + 3a2c2ce298...) =

61f7b61e1936f784f1899f1e8316c7a1b7540359e12de5197df4fea5f6a649c7

Level 2: Hash(1a6ad26337... + 61f7b61e19...) =

cef7841e35621474969c8d386a17ddcfbad6f006d5bc896c54baf45bde467d2f

Merkle Root:

cef7841e35621474969c8d386a17ddcfbad6f006d5bc896c54baf45bde467d2f

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.